Utility Maximization in Wireless Backhaul Networks with Service Guarantees

Nicholas Jones and Eytan Modiano
Laboratory for Information and Decision Systems, MIT
jonesn@mit.edu, modiano@mit.edu

***Abstract*—We consider the problem of maximizing utility in wireless backhaul networks, where utility is a function of satisfied service level agreements (SLAs), defined in terms of end-to-end packet delays and instantaneous throughput. We model backhaul networks as a tree topology and show that SLAs can be satisfied by constructing link schedules with bounded inter-scheduling times, an NP-complete problem known as pinwheel scheduling. For symmetric tree topologies, we show that simple round-robin schedules can be optimal under certain conditions. In the general case, we develop a mixed-integer program that optimizes over the set of admission decisions and pinwheel schedules. We develop a novel pinwheel scheduling algorithm, which significantly expands the set of schedules that can be found in polynomial time over the state of the art. Using conditions from this algorithm, we develop a scalable, distributed approach to solve the utility-maximization problem, with complexity that is linear in the depth of the tree.**

I. Introduction

Next-generation wireless networks must support increasing levels of real-time control and inference as these technologies become ubiquitous. In many such systems, the controller or learning model is located at an edge server, and the system relies on network communication to transport data from devices to the server in real time. Best-effort service is often insufficient for these systems, which demand hard guarantees on throughput and latency to operate safely and reliably.

In addition to hard service guarantees, these systems require ever-increasing data rates and power, which is a significant operating expense in 5G and is only expected to grow. At the same time, the cost of hardware deployment is dropping, incentivizing infrastructure providers to install more and more access points (APs) and small cells with wireless backhaul links. This creates a natural tree topology, with devices connected to APs at the leaves, and the server located at the root.

In this work we consider networks of this topology and uplink traffic with hard service requirements, dictated by service-level agremeents (SLAs) between the network operator (NO) and customers. We focus on uplink traffic because the data being sent to the model is typically larger than the model output, but the same analysis holds for downlink traffic. We assume that the NO receives some utility (revenue) from satisfying each SLA, which is a function of the service requirements it must satisfy, and we derive admission control and offline scheduling policies to maximize the NO's utility, while guaranteeing each SLA is satisfied. In particular, our policies are able to make hard guarantees on throughput and end-to-end packet delays. While we assume a wireless topology subject to interference, our results can be generalized to any TDMA backhaul network.

There is a large body of work on network utility maximization, which allocates rates to users to maximize utility and was first proposed by Kelly in [1]. A distributed solution to this problem was developed in [2]. It has been studied extensively in the context of stochastic network optimization, using Lyapunov-based approaches to maximize utility while ensuring network stability [3]. A summary of cross-layer techniques for wireless networks can be found in [4].

There has also been work on utility maximization with service guarantees. Hou and Kumar introduced a framework for meeting service requirements in single-hop wireless networks [5] and showed that this can be used to maximize utility in [6], where utility is determined by the fraction of user traffic which is served before its deadline. This framework was extended to multi-hop networks in [7].

Additionally, there is a growing body of work on scheduling in wireless backhaul networks. In [8], a throughput-maximizing scheduling policy is developed for wireless backhaul, and in [9] a backpressure-based policy is proposed with the same objective. In [10], a scheduling policy is designed which achieves at most twice the delay of an equivalent wired backhaul by multiplexing over both time and frequency. An online learning algorithm is used in [11] to minimize average wireless backhaul delay, while a cross-layer routing, scheduling, and resource allocation design is explored in [12].

Offline scheduling for throughput maximization in general wireless networks has also received considerable attention [13]–[15]. When deadline constraints are considered, optimal schedules can be found by relaxing interference and capacity constraints [16], [17]. Without these relaxations, the authors of [18], [19] show that packet delays can be bounded as a function of the schedule length by solving a mixed-integer program or using heuristic approaches.

Recently, a scheduling framework was proposed to make hard throughput and delay guarantees in wireless networks with interference constraints and general topologies [20]. Using a network slicing model, the authors show that tight deadline guarantees can be made by bounding link inter-scheduling times. Constructing schedules with bounded inter-

This material is based upon work supported by the Department of the Air Force under Air Force Contract No. FA8702-15-D-0001. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Department of the Air Force. This work was also supported in part by NSF grants CNS-2148183 and CNS-2148128.

978-1-6654-3176-7/22/$31.00 ©2022 IEEE

scheduling times is known as pinwheel scheduling and was first introduced in [21]. It is known to be NP-complete [22], but polynomial time algorithms exist which can form schedules under certain sets of conditions [23]–[26].

In this work, we build on the network slicing model and inter-scheduling conditions of [20] to make simultaneous throughput and delay guarantees for flows in a wireless backhaul network. We introduce a novel state-of-the-art algorithm called Inductive Pinwheel Scheduling (IPS) that significantly expands the set of pinwheel schedules that can be found in polynomial time. We derive feasibility conditions for pinwheel schedules based on this algorithm, and optimize over these scheduling conditions and flow admission decisions to efficiently maximize utility for a batch of flows, while guaranteeing that each flow's SLA is satisfied.

The remainder of this paper is organized as follows. In Section II, we introduce the system model. In Section III, we show that simple round-robin schedules can be optimal for symmetric trees, but a more complex policy based on pinwheel scheduling is required when asymmetry exists. In Section IV, we introduce the IPS algorithm and show that it constructs schedules in polynomial time which were previously unattainable. We incorporate the algorithm into our utility maximization problem as a mixed-integer program, and we present a scalable distributed algorithm for solving this program. In Section V, we provide numerical results for the IPS algorithm and our utility maximization framework. Note that proofs are omitted due to space constraints and are included in the technical report [27].

II. System Model

We consider a wireless network with a tree topology, denoted by $G = (V, E)$, where V is the set of nodes (vertices on the graph), and E is the set of links (edges on the graph). The tree consists of D levels, with the server (root node) at level 0 and the leaves (APs) at level $D-1$. Denote the set of level d nodes as V_d, and the level of node v as $d(v)$. Each node v has N_v children, and denote the set of its children as $\mathcal{C}_v$, the set of all its descendants as V_v^-, and the set of its ancestors as V_v^+. We consider uplink traffic so each node v has one outgoing directional link connecting it to its parent, which we denote as $e(v)$.

Time is slotted, with the duration of a time slot equal to the smallest schedulable interval dictated by the physical and link layers, and a unit of capacity as the capacity needed to send one packet over this time interval. Each link $e \in E$ has a fixed capacity c_e, which can vary between links. Links are error-free but subject to local interference, where each set of children that share a parent node mutually interfere, so only one child can be scheduled in each slot. Denote the set of links scheduled at time t as $\mu(t)$, and let $\mu_e(t) = 1$ if link e is scheduled at time t and 0 otherwise. Denote the maximum inter-scheduling time of link e, i.e., the largest continuous time interval between slots where link e is scheduled, as k_e. Let $\boldsymbol{k}$ be the vector of k_e values for all $e \in E$, and $\boldsymbol{k_v}$ be the vector of $k_{e(v')}$, for all $v' \in \mathcal{C}_v$. We use a superscript π to denote these quantities under a given scheduling policy π.

Network traffic takes the form of flows, where each flow corresponds to a customer connected wirelessly to an AP at level $D-1$. While connected, these customers form level D of the tree. We assume batch arrivals, where a batch of flows arrives simultaneously, and the network makes admission decisions and computes an offline schedule for the entire batch. Each customer establishes an SLA with the NO, dictating the level of service it requires. These SLAs are defined in terms of a maximum instantaneous arrival rate λ, a packet deadline τ, and an SLA duration T. For ease of exposition, we assume in this work that SLAs are equivalent for all flows. This can easily be extended to a flow-specific λ and τ values, which we will address in a future version of this work.

An SLA initialized at time t_0 dictates that if $\lambda_i(t) \leq \lambda$ packets belonging to f_i are generated at time t, each packet must be delivered to the server by time $t + \tau$, for every $t_0 \leq t < t_0 + T$. We assume the NO cannot accept an SLA unless it is guaranteed to be met. When a flow's SLA is satisfied, the NO receives some utility as a function of λ and τ. Because SLAs are symmetric, maximizing utility is equivalent to maximizing the number of supported SLAs, which we denote by σ. Note that this framework can also support flows which require guaranteed rates without hard deadlines, by setting τ to be large.

Denote flow f_i's route from its leaf to the root as $\mathcal{T}^{(i)}$, and the set of all leaf-to-root paths as $\mathcal{T}$. Each link $e \in \mathcal{T}^{(i)}$ is assigned a slice of capacity, which is reserved specifically for flow f_i. We denote the amount of capacity reserved for f_i, or the slice width, as $w_{i,e}$. In addition, each slice has its own dedicated queue, which decouples the queueing of different flows. We assume all scheduling policies are work-conserving, so when a link e is scheduled, it serves the smaller of the current queue size and the slice width of each flow.

A. Preliminary Results

In [20], the authors show that for any topology and interference constraints, using the same network slicing model described above, deadline guarantees can be made by satisfying conditions on maximum inter-scheduling times $\boldsymbol{k}$. Specifically, they show the following result for a given set of flows $\mathcal{F}$.

Lemma 1 ([20]). *If a scheduling policy exists which satisfies SLAs for all flows in $\mathcal{F}$, then there must exist an SLA-satisfying cyclic policy π with period K^π, such that*

$$\mu^\pi(t) = \mu^\pi(t + K^\pi),\ \forall\ t_0 \leq t < T - K^\pi. \tag{1}$$

Furthermore, if slice widths are set such that

$$w_{i,e} = \lambda k_e^\pi,\ \forall\ f_i \in \mathcal{F}, e \in \mathcal{T}^{(i)}, \tag{2}$$

then SLAs are satisfied for all flows $\mathcal{F}$ if

$$\sum_{e \in \mathcal{T}^{(i)}} k_e^\pi \leq \tau,\ \forall\ f_i \in \mathcal{F}, \tag{3}$$

$$\sum_{f_i \in \mathcal{F}: e \in \mathcal{T}^{(i)}} \lambda k_e^\pi \leq c_e,\ \forall\ e \in E. \tag{4}$$

This result shows that we can restrict ourselves to cyclic policies without loss of optimality, and shows how to construct an SLA-satisfying policy in terms of slice widths and inter-scheduling times. The deadline constraint (3) combined with the slice width condition (2) guarantees that all deadlines are met by forcing links to be scheduled somewhat regularly and queue sizes to remain small, while the slice capacity condition (4) ensures that a link has enough capacity to support the allocated slices. We encourage readers to reference [20] for more details.

Define the class of cyclic, work-conserving scheduling policies that satisfy our local interference constraints as Π_c. In order for queues to remain stable and for any finite deadline to be met as the SLA duration T goes to infinity, the service rate of each queue must be at least as large as the arrival rate [20]. Defining the time-average activation rate of link e under policy π as

$$\bar{\mu}_e^\pi \triangleq \frac{1}{K^\pi} \sum_{t=0}^{K^\pi - 1} \mu_e^\pi(t), \tag{5}$$

the queue stability condition can be written as

$$w_{i,e} \bar{\mu}_e^\pi \geq \lambda. \tag{6}$$

If the inter-scheduling time of link e is always the same under a policy π, i.e., it is scheduled *exactly* every k_e^π slots, then $k_e^\pi = 1/\bar{\mu}_e^\pi$, and the slice width condition (2) represents the smallest stable slice width under π. The larger k_e is relative to $1/\bar{\mu}_e$, the larger the slice width must be and the larger the minimum feasible τ becomes. This shows that schedules with regular inter-scheduling times are more efficient at satisfying SLAs. We will see this in more detail in the next section.

III. Utility Maximization on Symmetric Trees

Recall that our objective is to maximize the number of supported SLAs σ over all admission decisions and scheduling policies. We begin by analyzing this problem under symmetry conditions. Assume the tree G is perfectly symmetric at each level, such that each node v at level d has N_{d+1} children, and N_D flows arrive at each AP. Further assume that capacities are fixed across each level of the tree, with $c_{e(v)} = c_d$ for each node v at level d. To help analyze the problem in this setting, it is instructive to consider the opposite perspective and to define the feasible set of (λ, τ) pairs, such that *all* SLAs can be supported by some policy $\pi \in \Pi_c$ on G. Denote this region as $\Lambda(G)$, and define the throughput-optimal rate

$$\lambda^*(G) \triangleq \max_{(\lambda,\tau)\in\Lambda(G)} \lambda \tag{7}$$

as the largest value of λ in this set independent of τ. When $\lambda \leq \lambda^*(G)$, we say that λ is rate-feasible on G. Similarly define the delay-minimizing deadline

$$\tau^*(G) \triangleq \min_{(\lambda,\tau)\in\Lambda(G)} \tau \tag{8}$$

as the smallest value of τ in this set independent of λ. When $\tau \geq \tau^*(G)$ we say that τ is deadline-feasible on G. We will see that analyzing these quantities individually allows us to define $\Lambda(G)$ and provide insight into maximizing $|\mathcal{F}|$.

We first consider $\lambda^*(G)$, and assume without loss of generality that $\tau \geq DK^\pi$ for any policy π with finite period K^π. Then deadline constraints (3) are satisfied under any π that schedules each link at least once per scheduling period, and we can drop the deadline constraints in the analysis. To ensure feasibility as the SLA duration $T \to \infty$, we need only ensure that queue stability conditions (6) are satisfied and that allocated slice widths do not exceed link capacities. Setting slice widths to their smallest value while ensuring stability yields $w_{i,e(v)} = \lambda/\bar{\mu}_{e(v)}$, for each flow f_i and link e.

There are $|\mathcal{F}_v| = \prod_{d'=d+1}^{D} N_{d'}$ flows which pass through link $e(v)$, by the symmetry of G, so the capacity constraint at each link $e(v)$ is

$$\lambda \prod_{d'=d(v)+1}^{D} N_{d'} \leq \bar{\mu}_{e(v)} c_{e(v)}. \tag{9}$$

Then $\lambda^*(G)$ is the solution to the LP

$$\begin{aligned} \max_{\pi\in\Pi_c} \ & \lambda \\ \text{s.t. } & \lambda \prod_{d'=d(v)+1}^{D} N_{d'} \leq \bar{\mu}_{e(v)}^\pi c_{e(v)}, \ \forall\ v \in V_c, \\ & \sum_{v'\in\mathcal{C}_v} \bar{\mu}_{e(v')}^\pi \leq 1, \ \forall\ v \in V_c, \end{aligned} \tag{10}$$

where the second constraint ensures that no more than one link per set of children is scheduled per slot.

Lemma 2. *When G is perfectly symmetric at each level,*

$$\lambda^*(G) = \min_{0\leq d<D} \Big(\prod_{d'=d+1}^{D} N_{d'} \Big)^{-1} c_{d+1}, \tag{11}$$

and this is achieved by a universal round-robin (URR) scheduling policy, defined as a round-robin policy at each set of children.

Next we turn to analyzing $\tau^*(G)$, which is equivalent to the min-max delay seen by any packet. Again because $\tau^*(G)$ is defined independently of λ, we can assume without loss of generality that $\lambda = \epsilon$, for some strictly positive but arbitrarily small ϵ. Then the queue stability and slice capacity constraints are satisfied under any policy that schedules each link at least once per scheduling period, and to ensure feasibility we need only ensure that deadline constraints are met. Recall from (3) that a packet can meet any deadline $\tau \geq \sum_{e\in\mathcal{T}^{(i)}} k_e$. In other words, the maximum delay seen by any packet is upper bounded by the sum of inter-scheduling times along its route. Therefore,

$$\tau^*(G) \leq \max_{T\in\mathcal{T}} \sum_{e\in T} k_e \tag{12}$$

where recall that $\mathcal{T}$ denotes the set of all leaf-to-root paths in the tree.

Each route sees the same scheduling constraints at a given level of the tree due to the symmetry of G. Therefore,

minimizing the maximum inter-scheduling time at each level of the tree intuitively minimizes the upper bound in (12), because each route will see the same inter-scheduling times at each node. For a given node v, the solution to

$$\min_{\boldsymbol{k_v}} \max_{v' \in \mathcal{C}_v} k_{e(v')} \quad \text{s.t. } \boldsymbol{k_v} \text{ schedulable} \tag{13}$$

is given by a round-robin policy where $k_{e(v')} = |\mathcal{C}_v| = N_d$ for each v' at level d. It turns out that a URR policy not only minimizes the bound in (12), but that this bound is tight.

Lemma 3. *When G is perfectly symmetric at each level,*

$$\tau^*(G) = \sum_{d=1}^{D} N_d, \tag{14}$$

and both throughput- and deadline-optimality are achieved by a URR policy. Therefore,

$$\Lambda(G) = \{(\lambda, \tau) \mid \lambda \leq \lambda^*(G), \tau \geq \tau^*(G)\}, \tag{15}$$

and for any $(\lambda, \tau) \in \Lambda(G)$, a URR policy maximizes utility by supporting all SLAs on the tree G.

This lemma provides an exact characterization of $\Lambda(G)$ and shows that both $\lambda^*(G)$ and $\tau^*(G)$ can be jointly achieved by the same simple and intuitive URR policy. For (λ, τ) that lie within $\Lambda(G)$, this policy supports all flows and is by definition utility maximizing. This result also shows that, perhaps suprisingly, increasing the number of hops in a backhaul network actually decreases packet delay. If each flow was connected to a single AP, then the minimum achievable deadline would be the total number of flows $\prod_d N_d$. By introducing additional hops, the scheduling policy is able to mulitplex over both time and space, thereby decreasing τ^*.

A. Greedy Pruning

We next consider the case when $\tau < \tau^*(G)$, and the network cannot simultaneously satisfy deadline constraints for all flows. We will construct a subset of supported flows by pruning branches from the tree and cutting off all flows connected to these branches. When a leaf is pruned, this corresponds to cutting off a single flow, and when a branch farther up the tree is pruned, this corresponds to cutting off all flows whose APs belong to the subtree rooted at that branch node.

For a given tree G and deadline τ, let $\bar{\tau}(G) = \tau^*(G) - \tau$, which is the amount of time needed to be "cut" from $\tau^*(G)$ to achieve τ. Then consider the set of nodes V_{d-1} at level $d-1$. If a branch is pruned from each of these nodes, the value of N_d is decremented by one, and we refer to this as a level d prune. Note that we are pruning symmetrically across a level, so the resulting tree G' is still symmetric and Lemmas 2 and 3 still hold on G'. Denote the new value of N_d after pruning as $N_d(G')$. Then

$$\tau^*(G') = \sum_{d'=1}^{D} N_{d'}(G') = \sum_{d'=1}^{D} N_{d'} - 1 = \tau^*(G) - 1, \tag{16}$$

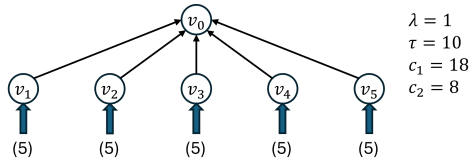

Fig. 1: Example tree with $D = 2$ and $N_D = 8$

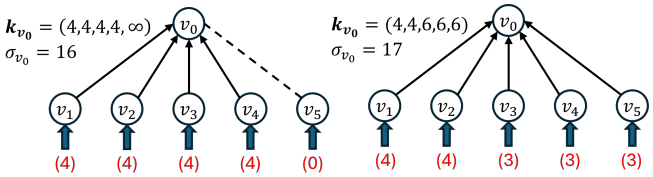

Fig. 2: Best URR policy (left) vs. optimal policy (right)

so performing $\bar{\tau}(G)$ of these pruning operations yields a tree G^* with $\tau^*(G^*) = \tau$. Note that $\tau^*(G^*)$ is independent of which levels are pruned.

Each level d prune cuts off $1/N_d$ of the flows connected to each node in V_{d-1}, so by symmetry it cuts off $1/N_d$ of all flows in the tree. Pruning at any level has the same effect on deadlines, but clearly pruning at level d as opposed to level d' cuts off fewer flows when $d > d'$. Based on this insight, we define a Greedy Pruning algorithm which performs $\bar{\tau}(G)$ sequential pruning operations, each time selecting a node to prune at any level $d \in \arg\max_d N_d(G')$, where G' is the current state of the tree at that operation. This algorithm is guaranteed to reach deadline feasibility while minimizing the number of flows which are pruned, producing the largest deadline-feasible subset of $\mathcal{F}$.

Theorem 1. *Let G be perfectly symmetric at each level, and G^* be the output of the Greedy Pruning algorithm on G. If $\lambda \leq \lambda^*(G^*)$, then $(\lambda, \tau) \in \Lambda(G^*)$, and a URR policy on G^* is utility-maximizing on G.*

From (11), we observe that λ^* can only increase as branches are pruned, so $\lambda^*(G^*) \geq \lambda^*(G)$, and even if λ is rate-infeasible on the original tree G, it may be rate-feasible on the Greedy Pruning output G^*. Unfortunately, if λ is not rate-feasible on G^*, there is no Greedy Pruning alternative that yields a utility-maximizing policy, and there may not exist a URR policy that is utility maximizing.

To see this, consider the depth-two tree G in Figure 1. From (14), we can immediately see that $\tau^*(G) = 10 \leq \tau$, so $\tau = 10$ is deadline-feasible on G. Next we use (11) to compute $\lambda^*(G) = 18/25 > \lambda$, so $\lambda = 1$ is rate-infeasible on G. One can easily verify that pruning once at either level of the tree is insufficient to make λ rate-feasible, so we perform two pruning operations. Pruning two branches at either level yields $\lambda^*(G^*) = 18/15$ and $\sigma = 15$, while pruning one branch at each level yields $\lambda^*(G^*) = 18/16$ and $\sigma = 16$, making it the optimal URR policy. We show the pruned tree and this policy on the left of Figure 2.

On the right we show the optimal policy for the tree G and note that it is not a URR policy. By pruning asymmetrically

and deviating from URR, we are able to support an additional flow. In particular, let the flows at each AP be scheduled round-robin and the level-1 APs follow the schedule

$$\pi = \{v_1, v_3, v_2, v_4, v_1, v_5, v_2, v_3, v_1, v_4, v_2, v_5\}, \tag{17}$$

which has a length of 12 slots and repeats itself indefinitely. One can verify that this satisfies the vector of inter-scheduling times in Figure 2, and that both slice capacity and deadline constraints are met. This example shows the need for a more complete framework to maximize utility under general conditions. In the subsequent sections we will develop this framework, which can handle not only rate-infeasibililty on symmetric trees, but also asymmetric topologies and arrivals.

IV. General Utility Maximization

Let σ_v^π be the number of supported flows in the subtree rooted at node v under policy π. Using the conditions in Lemma 1, the general utility maximization problem can be written as the following integer program, where $\tilde{\mathcal{T}}$ represents the set of leaf-to-root paths which are not pruned by the admission decisions.

$$\begin{aligned}
(P1) \ : \ \max_{\pi \in \Pi_c, \tilde{\mathcal{T}} \subseteq \mathcal{T}} \ & \sigma^\pi \\
\text{s.t. } & \sigma_v^\pi = \sum_{v' \in \mathcal{C}_v} \sigma_{v'}^\pi, \ \forall \ v \in V, \\
& \sigma_v^\pi \leq N_v, \ \forall \ v \in V_{D-1}, \\
& \sigma_v^\pi \lambda k_{e(v)}^\pi \leq c_{e(v)}, \ \forall \ v \in V, \\
& \sum_{e \in T} k_e^\pi \leq \tau, \ \forall \ T \in \tilde{\mathcal{T}}, \\
& \boldsymbol{k}_v^\pi \text{ schedulable}, \ \forall \ v \in V, \\
& \sigma_v^\pi, k_{e(v)}^\pi \in \mathbb{Z}_+, \ \forall \ v \in V.
\end{aligned} \tag{18}$$

The first constraint is a flow conservation condition, the second ensures that we can support no more flows than are present at each AP, the third is the slice capacity constraint, and the fourth is the deadline constraint. Before addressing how to solve this program, we first must explore the schedulability constraint for each set of inter-scheduling times $\boldsymbol{k}_v$.

A. Inductive Pinwheel Scheduling

As discussed in the introduction, forming a schedule with constraints on maximum inter-scheduling times is known as pinwheel scheduling, and determining whether such a schedule exists for a given $\boldsymbol{k}$ is NP-complete [21]. Polynomial-time algorithms exist which can schedule certain classes of vectors, and the state-of-the-art algorithm S_{xy} is able to schedule all known polynomial-time schedulable vectors [23].

The schedulability of a vector is closely linked to a quantity known as its density and defined as $\rho(\boldsymbol{k}) \triangleq \sum_i 1/k_i$. A necessary condition for schedulability is that a vector's density is no more than 1, and vectors with lower density are often easier to schedule, though this is not always true. The S_{xy} algorithm can schedule at least all vectors with densities up to 0.7, and all vectors with densities up to $5/6$ are known to be schedulable, though not in polynomial time [28].

We introduce a novel pinwheel scheduling algorithm called *Inductive Pinwheel Scheduling (IPS)* that significantly expands the set of polynomial-time schedulable vectors. The algorithm is based on a surprisingly simple technique of removing elements from the vector one by one and augmenting the remaining values, such that if the augmented vector is schedulable then the original vector must also be schedulable. Due to space constraints, we omit the details and encourage readers to reference the technical report [27].

Theorem 2. *Denote the set of vectors schedulable by a pinwheel scheduling algorithm $\mathcal{A}$ as $\mathcal{K}_\mathcal{A}$. Then $\mathcal{K}_{IPS} \supset \mathcal{K}_\mathcal{A}$ for any known polynomial-time algorithm $\mathcal{A}$.*

While it is difficult to characterize $\mathcal{K}_{IPS}$ or provide general density guarantees beyond the 0.7 guarantee given by S_{xy}, numerical results in Section V show that IPS significantly outperforms S_{xy} on over 1.4 million randomly generated vectors with densities between 0.7 and 1. Most notably, *every vector* with density up to the achievability bound $5/6$ was scheduled by IPS, which leads us to the following conjecture.

Conjecture 1. *The IPS algorithm can schedule all vectors with densities up to $5/6$.*

The improvement in schedulable density from IPS can lead to a direct increase in achievable utility in $(P1)$. Recall that the solution is truly optimal only as far as we can optimize over all feasible schedules. By scheduling a larger class of high-density vectors, IPS shrinks this optimality gap and helps ensure that utility is limited by the network itself and not by scheduling limitations. We can incorporate the algorithm into the optimization directly by introducing an additional variable for each variable used in the algorithm, along with constraints describing the variables' relationships. Again due to space constraints we omit the details of the formulation and include them in the technical report [27].

After formulating and linearizing constraints, we are left with a mixed-integer linear program (MILP) that is equivalent to $(P1)$ under IPS schedulability constraints. This MILP has $O\big(\sum_{v \in V_c} M_v^2\big)$ binary variables and $O\big(\sum_{v \in V_c} M_v^2\big)$ linear constraints, where V_c is the set of vertices in the tree with children, i.e., all vertices except the leaves. Modern mixed-integer solvers like Gurobi have become quite powerful at solving well-structured MILPs with several hundred binary variables, with typical solve times ranging from less than a second to a few minutes on a standard laptop. Worst-case complexity, however, is always exponential in the number of binary variables, and the solve times can also grow exponentially. As a result, solving this MILP directly is only practical on small problem instances, where V_c cannot grow too large.

We develop a scalable solution to the problem by decomposing the global optimization into a series of subproblems at each node v, where each subproblem has a fixed $O(M_v^2)$ binary variables and $O(M_v^2)$ linear constraints. These subproblems are tractable for reasonable values of M, and the total solve time grows linearly, rather than exponentially, in the size of V_c. We explore this decomposition next.

B. Distributed SLA Utility Maximization

The bulk of the problem complexity lies in optimizing over the IPS schedulability constraints, which are independent between nodes under our local interference model. A close examination of $(P1)$ shows that the $\boldsymbol{k_v}$ vectors are coupled only through the deadline constraints and the flow conservation constraints. We will show that these can also be decoupled by solving for the maximum flow at each node independently.

Assume that deadline constraints are relaxed and consider flow conservation. The number of flows σ_v^* which pass through node v under a utility-maximizing policy is a function of the scheduling decisions along each leaf-to-root path that contains v, so this quantity is coupled between nodes. Now define the maximum number of flows supported by the subtree rooted at node v, independent of the rest of the tree, as

$$\hat{\sigma}_v^* = \max_{\boldsymbol{k_v}} \sum_{v' \in \mathcal{C}_v} \sigma_{v'}, \tag{19}$$

subject to $\sigma_{v'} \leq \hat{\sigma}_{v'}^*$, for all $v' \in \mathcal{C}_v$, and slice capacity and IPS constraints. Note the distinction between this quantity and σ_v^*. The former represents an upper bound on the latter, and is independent of scheduling decisions above it in the tree.

Now consider the deadline constraints $\sum_{e \in T} k_e \leq \tau,\ \forall\, T \in \mathcal{T}$. At first glance, these constraints appear to pose a serious challenge to our decomposition because they directly couple the inter-scheduling times along each leaf-to-root path. The key observation is that only the sum $\sum_{v' \in V_v^+} k_{e(v')}$ affects $\hat{\sigma}_v^*$, where recall that V_v^+ denotes the set of node v's ancestors, and the individual values of each $k_{e(v')}$ are irrelevant.

To see this, define the quantity

$$\tau_v \triangleq \tau - k_{e(v)} - \sum_{v' \in V_v^+} k_{e(v')}, \tag{20}$$

which represents the effective deadline by which packets must reach node v. When τ_v is fixed, the deadline constraints can be rewritten as

$$\sum_{v' \in T \cap V_v^-} k_{e(v')} \leq \tau_v,\ \forall\, T \in \mathcal{T}, \tag{21}$$

which shows that $\boldsymbol{k_v}$ only depends on the quantity τ_v and vectors $\boldsymbol{k_{v'}}$ below it in the tree.

Therefore, when τ_v is fixed, $\hat{\sigma}_v^*$ can be computed in the following way, using only knowledge of the scheduling decisions below node v in the tree. We use the notation $\hat{\sigma}_v^*(\tau_v)$ to show the dependence on τ_v.

Lemma 4. *Let τ_v and $\hat{\sigma}_{v'}^*(\tau_{v'})$ be fixed, for all $v' \in \mathcal{C}_v$ and all feasible $\tau_{v'}$. Then $\hat{\sigma}_v^*(\tau_v)$ is the solution to*

$$\begin{aligned}(P2): \max_{\boldsymbol{k_v}, \pi_v} & \sum_{v' \in \mathcal{C}_v} \sigma_{v'} \\ \text{s.t. } & \sigma_{v'} \leq \hat{\sigma}_{v'}^*(\tau_v - k_{e(v')}),\ \forall\, v' \in \mathcal{C}_v, \\ & \sigma_{v'} \lambda k_{e(v')} \leq c_{e(v')},\ \forall\, v' \in \mathcal{C}_v, \\ & \pi_v = IPS(\boldsymbol{k_v}), \\ & \sigma_{v'}, k_{e(v')} \in \mathbb{Z}_+,\ \forall\, v' \in \mathcal{C}_v.\end{aligned} \tag{22}$$

Note the differences between $(P1)$ and $(P2)$. While $(P1)$ is a global optimization that solves for σ_v^* simultaneously at each v, $(P2)$ is a local optimization, which solves for a particular $\hat{\sigma}_v^*$. The slice capacity and IPS constraints are the same because these constraints are decoupled between nodes, while the deadline constraints from $(P1)$ are implicitly present in the $(P2)$ flow conservation constraints $\sigma_{v'} \leq \hat{\sigma}_{v'}^*(\tau_v - k_{e(v')}),\ \forall\, v' \in \mathcal{C}_v$. To see this, observe that when τ_v and $k_{e(v')}$ are fixed, $\tau_{v'} = \tau_v - k_{e(v')}$, and the constraint becomes $\sigma_{v'} \leq \hat{\sigma}_{v'}^*(\tau_{v'})$, which enforces both flow conservation and deadlines.

Solving $(P2)$ at node v requires knowledge of $\hat{\sigma}_{v'}^*$ at its children, which naturally lends itself to an algorithm which begins at the APs and iterates up the tree until reaching the root. However, because τ_v depends on the scheduling decisions at nodes *above* v in the tree, the algorithm cannot know the optimal value τ_v^* a priori. Instead, it solves $(P2)$ once for each feasible value of τ_v, and returns the set

$$\{\hat{\sigma}_v^*(\tau_v),\ \forall\ D - d(v) \leq \tau_v \leq \tau - d(v)\} \tag{23}$$

to its parent, which forms the upper bound in the flow conservation constraint at its parent node. This encompasses all feasible values of τ_v because $k_{e(v')} \geq 1$ at each $v' \in V$, so the algorithm must solve $(P2)$ a total of $\tau - D$ times at each node.

Once the algorithm has solved $(P2)$ at the root, it propagates the optimal solution back down the tree by iteratively finding $\hat{\sigma}_v^*(\tau_v^*)$ from the pre-computed values at node v, and then setting $\tau_{v'}^* = \tau_v^* - k_{e(v')}^*$ for each $v' \in \mathcal{C}_v$.

While $(P2)$ contains non-linear and non-convex constraints, it can be linearized in the same way as $(P1)$, resulting in a MILP with $O(M_v^2)$ binary variables and $O(M_v^2)$ linear constraints. The algorithm then solves at most $(\tau - D)V_c$ iterations of this linearized version to reach the global optimum. We can reduce the dependency on V_c even further, however, by solving in a distributed fashion. Because each subproblem only depends on values below it in the tree, we can parallelize the computations across each level, resulting in a total computation time that is linear in the tree depth D rather than V_c. We describe the full Distributed SLA Utility Maximization ($DSUM$) algorithm in Algorithm 1.

Theorem 3. *$DSUM$ returns the solution to $(P1)$, maximizing utility over all admission decisions and IPS scheduling policies. It does this by setting*

$$\hat{\sigma}_v^*(\tau_v) = \min\left\{N_{D,v}, \tau_v, \left\lfloor \frac{c_D}{\lambda} \right\rfloor\right\} \tag{24}$$

at each AP v, and for all feasible τ_v, and then solving at most $\tau(D-1)$ sequential batches of $(P2)$, where each batch consists of parallel $(P2)$ computations across nodes on one level of the tree.

The $DSUM$ algorithm reduces the computation from solving a MILP with $O(V_c M^2)$ binary variables to solving at most $\tau(D-1)$ batches of parallel MILPs with $O(M^2)$ binary variables each. In the next section, we will show numerical results including computation times, but in general $DSUM$

Algorithm 1: Distributed SLA Utility Maximization (DSUM)

Input: Flow arrivals $N_{D,v},\ \forall\ v \in V_{D-1}$, rate λ, deadline τ
Output: Maximum utility $\hat{\sigma}^*$, optimal policy π^*

for $v \in V_{D-1}$ **do**
  for $\tau_v \in [1, \tau - D + 1]$ **do**
    Set $\hat{\sigma}_v^*(\tau_v) = \min\left\{N_{D,v}, \tau_v, \left\lfloor \frac{c_D}{\lambda} \right\rfloor\right\}$
$d = D - 2$
while $d > 0$ **do**
  for *parallel* $v \in V_d$ **do**
    for $\tau_v \in [1, \tau - d]$ **do**
      Set $\hat{\sigma}_v^*(\tau_v), \boldsymbol{k_v^*}(\tau_v)$ from $(P2)$
  Decrement d
Set $\hat{\sigma}_{root}^*(\tau)$ from $(P2)$
$\tau_{root}^* = \tau$
while $d < D$ **do**
  for *parallel* $v \in V_d$ **do**
    for $v' \in \mathcal{C}_v$ **do**
      $k_{e(v')}^* = k_{e(v')}^*(\tau_v^*)$
      $\tau_{v'}^* = \tau_v^* - k_{e(v')}^*$
    $\pi_v = IPS(\boldsymbol{k_v})$
  Increment d
Return $(\hat{\sigma}_{root}^*, \pi^* = \{\pi_v,\ \forall\ v \in V_c\})$

can be solved on the order of minutes for reasonable values of M. As such, $DSUM$ offers a practical and provably near-optimal solution to maximizing utility with long-lasting SLAs.

V. Numerical Results

A. IPS Algorithm

In this section we present numerical results showing the performance of the IPS and $DSUM$ algorithms. To quantify the improvement of the IPS algorithm over S_{xy}, we randomly generated a large number of vectors and tested the performance of both algorithms on each one. These vectors varied in length from 4 to 20 elements. For each length M, we randomly generated $100,000$ unique vectors with densities between 0.7 and 1. The average success rate of both algorithms is shown on the left of Figure 3. For $M = 4$, IPS did not present any improvement over S_{xy}, but starting at length 5 the performance gap widened, and for $M \geq 8$, IPS consistently scheduled from 19% to 24% more vectors than S_{xy}.

On the right of Figure 3 we show the minimum density which was not schedulable for each vector length, and see that for $M \geq 7$, the minimum IPS density was consistently 0.05 to 0.07 higher than S_{xy}. In addition, out of over 1.4 million vectors that were tested, the smallest density which was not schedulable by IPS was 0.834. This supports our conjecture that IPS can schedule all vectors with density up to $5/6 \approx 0.833$, which recall is the largest possible density bound. To further support this conjecture, we repeated the experiment with 1.3 million unique random vectors with densities in the range $(0.7, 0.83]$. As expected, IPS was able to schedule every one.

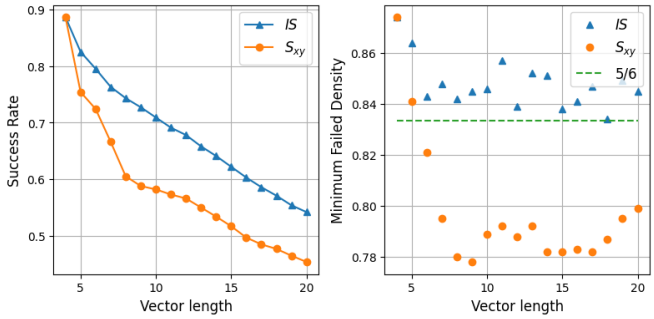

Fig. 3: Success rate and minimum unschedulable density of the IPS and S_{xy} algorithms

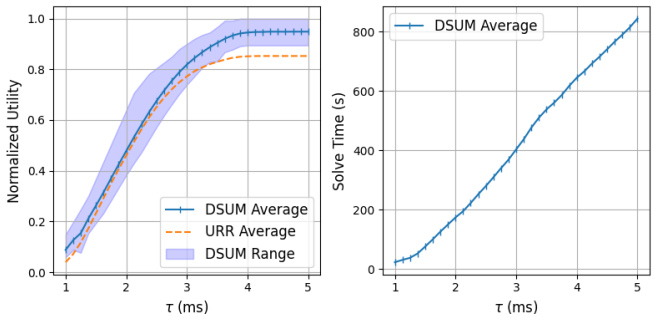

Fig. 4: Normalized utility and solve times for the $DSUM$ algorithm on trees with depth 3 and varying degrees from 2 to 6, with 400 flows requesting SLAs

B. $DSUM$ Algorithm

We tested the performance of the $DSUM$ algorithm on various trees of depth 3, while varying the link capacities, the number of children (i.e., the degree) at each node, and the number of flows at each AP. For the purposes of these simulations, we set $\lambda = 10$ Mbps for each flow, and the duration of a time slot as 125 μs, the smallest time slot used for data in the 5G standard [29]. Note that both λ and the time slot duration can be scaled without changing the results, provided the link capacities are scaled with λ.

For each experiment, we constructed a depth 3 tree with a random number of children at each node, uniformly distributed on the interval $[2, 6]$. The uplink capacity from each user to its AP was fixed at 250 Mbps, and the maximum capacities of the level-2 and level-1 links were fixed at 1 Gbps and 4 Gbps respectively. When each tree was constructed, the actual capacity of each level-2 and level-1 link was drawn uniformly from the range $[C_{max}/2, C_{max}]$, where C_{max} represents the maximum capacity for the link as described above. We fixed the number of flows requesting SLAs at 400, and randomly distributed them at the APs following a uniform distribution.

Once each tree was constructed, we ran the $DSUM$ algorithm on the tree and observed the utility the algorithm was able to achieve, as well as the solve time of the algorithm. We also tested the performance of the URR algorithm with greedy pruning. The results of 50 such experiments are shown in Figure 4 as a function of τ. The left side of the figure

shows the utility averaged over each experiment, as well as the maximum and minimum values, with all utilities normalized by the tree capacity. Note that the capacity is an upper bound on utility, and may not be achievable for bounded deadlines and schedule length. Nevertheless, we observe that the average $DSUM$ performance is approximately 0.95 of this bound as τ becomes large, and achieves the bound with equality in some instances. When τ is small, more branches must be pruned, and greedy pruning with URR yields near-symmetric conditions, making it close to optimal. As τ increases, we observe that $DSUM$ is able to achieve approximately 10% higher utility than URR.

The right side of Figure 4 shows the average solve times for $DSUM$ as a function of τ, averaged over the same set of experiments. These experiments were run on a standard laptop with an Intel Core i7-1365U processor. The solve times increase roughly linearly with τ as expected, because each node solves approximately τ versions of $(P2)$. We observe that solve times for the complete algorithm are on the order of minutes for depth 3 trees with degree up to 6 and 400 SLAs.

VI. Conclusion

In this paper, we studied the problem of maximizing utility from SLAs, defined in terms of instantaneous throughput and hard packet deadlines. We showed that under perfect symmetry conditions, a simple URR policy is optimal. We introduced the IPS algorithm, which significantly outperforms the state of the art, and formulated a MILP with conditions from this algorithm to solve the utility maximization problem. We then developed a scalable distributed algorithm to solve this MILP, with solve times that grow linearly in the depth of the tree.

References

[1] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, "Rate control for communication networks: shadow prices, proportional fairness and stability," *Journal of the Operational Research society*, vol. 49, no. 3, pp. 237–252, 1998.

[2] S. H. Low and D. E. Lapsley, "Optimization flow control. i. basic algorithm and convergence," *IEEE/ACM Transactions on networking*, vol. 7, no. 6, pp. 861–874, 1999.

[3] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.

[4] L. Georgiadis, M. J. Neely, L. Tassiulas *et al.*, "Resource allocation and cross-layer control in wireless networks," *Foundations and Trends® in Networking*, vol. 1, no. 1, pp. 1–144, 2006.

[5] I.-H. Hou, V. Borkar, and P. R. Kumar, "A Theory of QoS for Wireless," in *IEEE INFOCOM 2009*. Rio de Janeiro, Brazil: IEEE, Apr. 2009, pp. 486–494. [Online]. Available: https://ieeexplore.ieee.org/document/5061954/

[6] I.-H. Hou and P. Kumar, "Utility-optimal scheduling in time-varying wireless networks with delay constraints," in *Proceedings of the eleventh ACM international symposium on Mobile ad hoc networking and computing*, 2010, pp. 31–40.

[7] R. Li and A. Eryilmaz, "Scheduling for end-to-end deadline-constrained traffic with reliability requirements in multihop networks," *IEEE/ACM Transactions on Networking*, vol. 20, no. 5, pp. 1649–1662, 2012.

[8] A. Kabbani, T. Salonidis, and E. W. Knightly, "Distributed low-complexity maximum-throughput scheduling for wireless backhaul networks," in *IEEE INFOCOM 2007 - 26th IEEE International Conference on Computer Communications*, 2007, pp. 2063–2071.

[9] S. Gopalam, S. V. Hanly, and P. Whiting, "Distributed and local scheduling algorithms for mmwave integrated access and backhaul," *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1749–1764, 2022.

[10] G. Narlikar, G. Wilfong, and L. Zhang, "Designing multihop wireless backhaul networks with delay guarantees," *Wireless Networks*, vol. 16, no. 1, pp. 237–254, 2010.

[11] Z. Han, H. Tan, R. Wang, S. Tang, and F. C. Lau, "Online learning based uplink scheduling in hetnets with limited backhaul capacity," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 2348–2356.

[12] M. Cao, X. Wang, S.-J. Kim, and M. Madihian, "Multi-hop wireless backhaul networks: A cross-layer design paradigm," *IEEE Journal on Selected Areas in Communications*, vol. 25, no. 4, pp. 738–748, 2007.

[13] K. Jain, J. Padhye, V. N. Padmanabhan, and L. Qiu, "Impact of interference on multi-hop wireless network performance," in *Proceedings of the 9th annual international conference on Mobile computing and networking*, 2003, pp. 66–80.

[14] B. Hajek and G. Sasaki, "Link scheduling in polynomial time," *IEEE Transactions on Information Theory*, vol. 34, no. 5, pp. 910–917, Sep. 1988.

[15] M. Kodialam and T. Nandagopal, "Characterizing achievable rates in multi-hop wireless networks: the joint routing and scheduling problem," in *Proceedings of the 9th annual international conference on Mobile computing and networking*, 2003, pp. 42–54.

[16] R. Singh and P. Kumar, "Throughput optimal decentralized scheduling of multihop networks with end-to-end deadline constraints: Unreliable links," *IEEE Transactions on Automatic Control*, vol. 64, no. 1, pp. 127–142, 2018.

[17] ——, "Adaptive csma for decentralized scheduling of multi-hop networks with end-to-end deadline constraints," *IEEE/ACM Transactions on Networking*, vol. 29, no. 3, pp. 1224–1237, 2021.

[18] P. Djukic and S. Valaee, "Quality-of-service provisioning for multi-service tdma mesh networks," in *Managing Traffic Performance in Converged Networks: 20th International Teletraffic Congress, ITC20 2007, Ottawa, Canada, June 17-21, 2007. Proceedings*. Springer, 2007, pp. 841–852.

[19] ——, "Delay Aware Link Scheduling for Multi-Hop TDMA Wireless Networks," *IEEE/ACM Transactions on Networking*, vol. 17, no. 3, pp. 870–883, Jun. 2009.

[20] N. Jones and E. Modiano, "Optimal Slicing and Scheduling with Service Guarantees in Multi-Hop Wireless Networks," in *Proceedings of the Twenty-fifth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. Athens Greece: ACM, Oct. 2024, pp. 181–190.

[21] R. Holte, A. Mok, L. Rosier, I. Tulchinsky, and D. Varvel, "The pinwheel: A real-time scheduling problem," in *Proceedings of the 22nd Hawaii International Conference of System Science*, 1989, pp. 693–702.

[22] A. Mok, L. Rosier, I. Tulchinsky, and D. Varvel, "Algorithms and complexity of the periodic maintenance problem," *Microprocessing and Microprogramming*, vol. 27, no. 1-5, pp. 657–664, Aug. 1989.

[23] M. Chan and F. Chin, "General schedulers for the pinwheel problem based on double-integer reduction," *IEEE Transactions on Computers*, vol. 41, no. 6, pp. 755–768, Jun. 1992. [Online]. Available: http://ieeexplore.ieee.org/document/144627/

[24] M. Y. Chan and F. Chin, "Schedulers for larger classes of pinwheel instances," *Algorithmica*, vol. 9, no. 5, pp. 425–462, May 1993.

[25] C.-C. Han, K.-J. Lin, and C.-J. Hou, "Distance-constrained scheduling and its applications to real-time systems," *IEEE Transactions on Computers*, vol. 45, no. 7, pp. 814–826, 1996.

[26] S.-S. Lin and K.-J. Lin, "A Pinwheel Scheduler for Three Distinct Numbers with a Tight Schedulability Bound," *Algorithmica*, vol. 19, no. 4, pp. 411–426, Dec. 1997.

[27] N. Jones and E. Modiano, "Utility maximization in wireless backhaul networks with service guarantees," 2026. [Online]. Available: https://arxiv.org/abs/2601.01630

[28] A. Kawamura, "Proof of the Density Threshold Conjecture for Pinwheel Scheduling," in *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. Vancouver BC Canada: ACM, Jun. 2024, pp. 1816–1819. [Online]. Available: https://dl.acm.org/doi/10.1145/3618260.3649757

[29] 3rd Generation Partnership Project (3GPP), "Nr; physical channels and modulation," ETSI, Tech. Rep. TS 138 211 V17.5.0, dec 2023, release 17. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/138200_138299/138211/